

Magíster en Ciencia de Datos
para la Innovación

FUNDAMENTOS DE BASES DE DATOS Y ALGORITMOS: Parte 1

JOSÉ FUENTES

Ingeniero Civil Informático

Doctor en Ciencias de la Computación

jfuentess@inf.udec.cl

CLASE #6

Análisis de algoritmos



Facultad de Ingeniería
Universidad de Concepción



CENTRO DE DATOS E
INTELIGENCIA ARTIFICIAL
UNIVERSIDAD DE CONCEPCIÓN

¿Qué tan costoso es ...

... buscar un elemento en la lista A?

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A =	5	8	1	-5	2	4	-1	0	9	-3	7	-9	3	-7	-4	-6

¿Buscar el elemento 5?

¿Buscar el elemento 2?

¿Buscar el elemento -6?

¿Cuál es el peor caso?

Notación Python

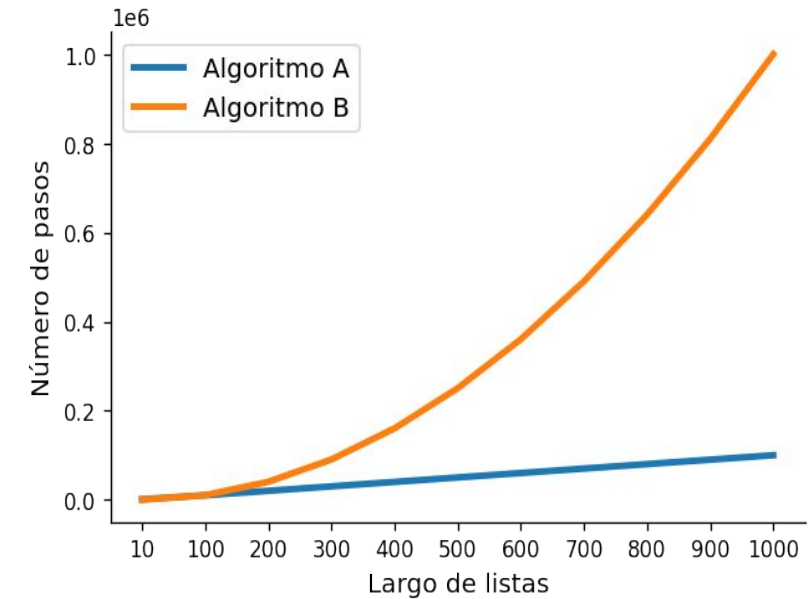
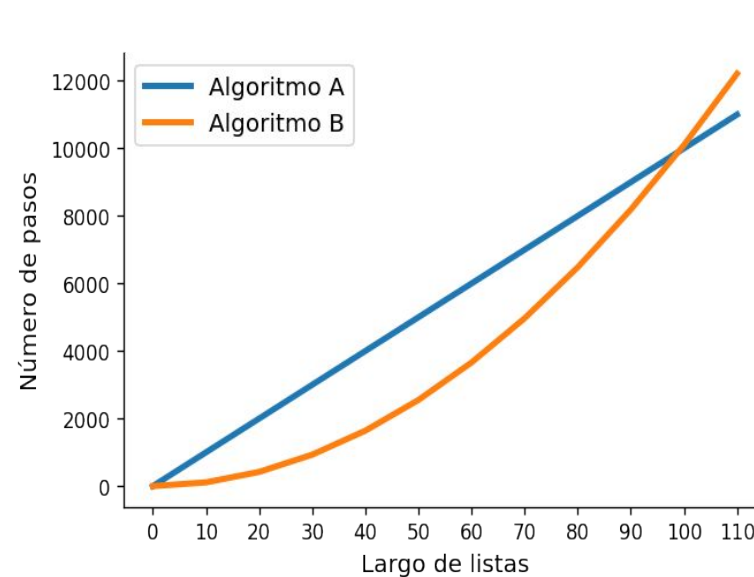
```
# Retorna True si x está  
# en la lista A  
  
def buscar(A, x)  
    for e in A:  
        if e == x:  
            return True  
    return False
```

Análisis de algoritmos: una introducción

Ejemplo: Tenemos un problema que involucra procesar una lista de n elementos. El problema puede ser resuelto por medio de dos algoritmos A y B.

- El algoritmo A toma $100n+1$ pasos para terminar
- El algoritmo B toma n^2+n+1 pasos para terminar

n	Tiempo algoritmo A	Tiempo algoritmo B
10	1 001	111
100	10 001	10 101
1 000	100 001	1 001 001
10 000	1 000 001	100 010 001



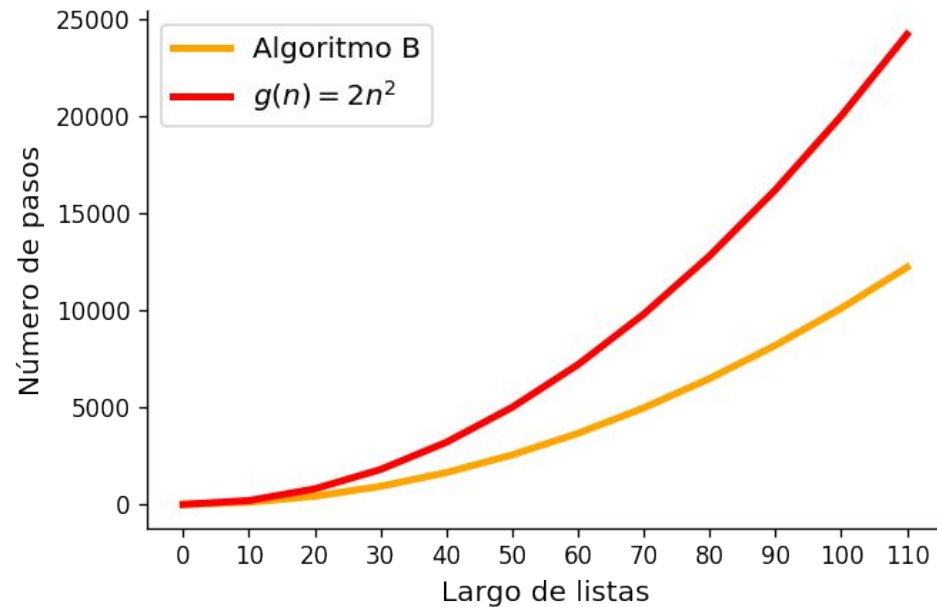
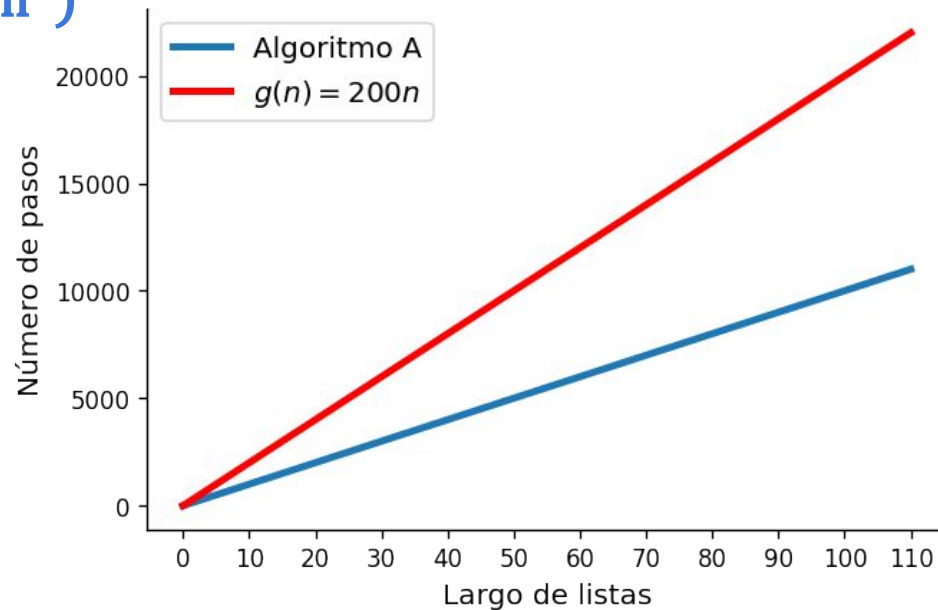
Análisis de algoritmos: una introducción

Límite o cota superior

$f(n) = O(g(n))$ si y sólo si $\exists c$ y n_0 , tal que $0 \leq f(n) \leq cg(n)$, para $c > 0$ y $n \geq n_0$

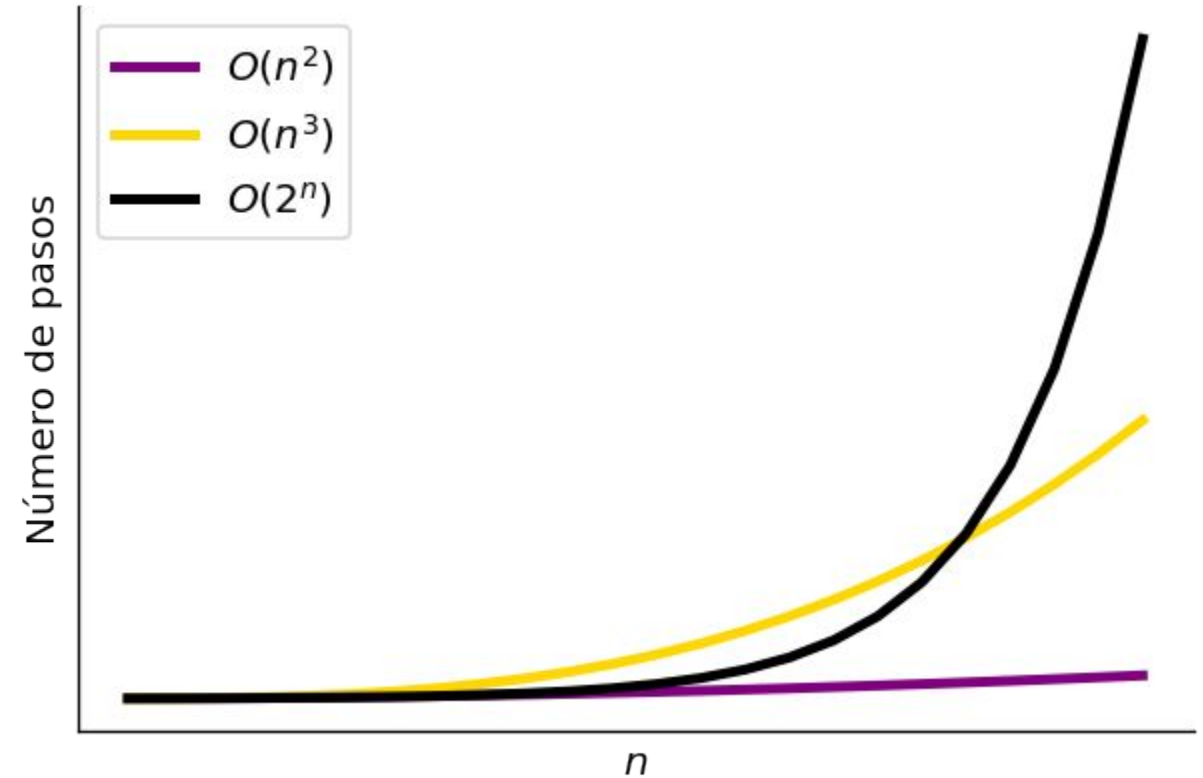
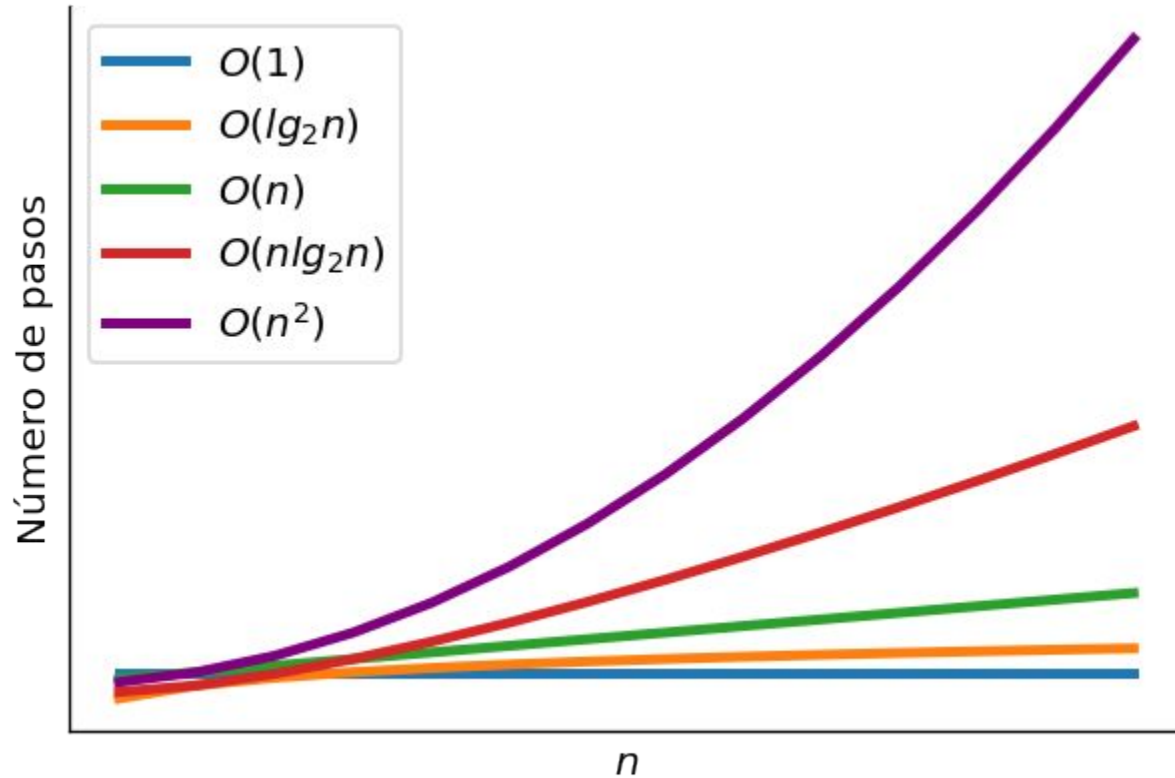
Algoritmo A : $100n+1 = O(n)$

Algoritmo B : $n^2+n+1 = O(n^2)$



Análisis de algoritmos: una introducción

Funciones clásicas



Análisis de algoritmos: una introducción

Búsqueda en una lista ordenada

Búsqueda lineal

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
-9	-7	-6	-5	-4	-3	-1	0	1	2	3	4	5	7	8	9

Búsqueda binaria

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
-9	-7	-6	-5	-4	-3	-1	0	1	2	3	4	5	7	8	9

Análisis de algoritmos: una introducción

Ordenamiento de una lista: selection sort

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
5	8	1	-5	2	4	-1	0	9	-3	7	-9	3	-7	-4	-6

Análisis

5	8	1	-5	2	4	-1	0	9	-3	7	-9	3	-7	-4	-6
-9	8	1	-5	2	4	-1	0	9	-3	7	5	3	-7	-4	-6
-9	-7	1	-5	2	4	-1	0	9	-3	7	5	3	8	-4	-6
-9	-7	-6	-5	2	4	-1	0	9	-3	7	5	3	8	-4	1
-9	-7	-6	-5	2	4	-1	0	9	-3	7	5	3	8	-4	1
-9	-7	-6	-5	-4	4	-1	0	9	-3	7	5	3	8	2	1
-9	-7	-6	-5	-4	-3	-1	0	9	4	7	5	3	8	2	1

...

Análisis de algoritmos: una introducción

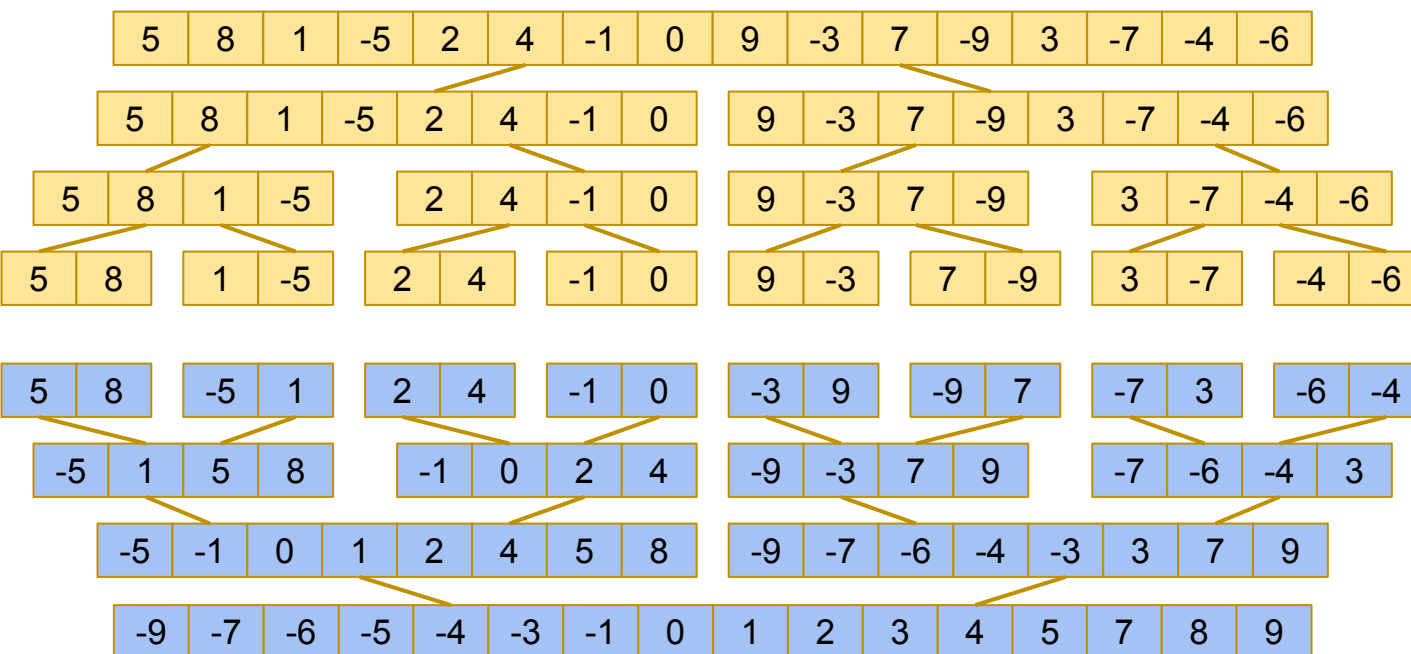
Ordenamiento de una lista: merge sort

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
5	8	1	-5	2	4	-1	0	9	-3	7	-9	3	-7	-4	-6

Análisis

Dividir

mezclar



Análisis de algoritmos: una introducción

¿Cómo se ve en Python?

Notación Python

```
# Lista L
n = len(L)
for x in L:
    print(x)
```

$O(n)$

Notación Python

```
# Suma de los n primeros
# enteros
x = (n*(n+1)) / 2
```

$O(1)$

Notación Python

```
# Listas L y M
n = len(L)
m = len(M)
for x in L:
    for y in M:
        print(x, y)
```

$O(nm)$

Notación Python

```
# Valor entero n
while n > 0:
    print(n)
    n = n // 2
```

$O(\lg_2 n)$

¡A PRACTICAR!

